Universidad San Carlos de Guatemala

Organización de Lenguajes y Compiladores 2

Ing. Mario Tobar



MANUAL USUARIO

Mario Raúl Yancor Ocaña

201930761

Quetzaltenango, 27 de octubre de 2026

Presentación del sistema

Codex Latinus fue la aplicación inicial que permitió escribir o abrir programas Latinus, ejecutar análisis léxico, sintáctico y semántico, consultar el AST, revisar la tabla de símbolos, navegar por una pila de procesos y convertir un programa válido a PigLatin.

En el Proyecto 2, esa base evolucionó a Contacto 3xtrat3rr3str3D. La aplicación trabaja con carpetas que pueden contener archivos Y? (.y), Zetariano (.z) y PigLatin (.pig), realiza el análisis del proyecto y, cuando es válido, genera cuartetas, una representación de memoria/C3D, código C y código ensamblador RISC-V.

Funciones principales

Proyecto 1: crear, abrir y guardar archivos Latinus (.lat), analizar errores y visualizar AST, tabla de símbolos y pila SHIFT / REPLACE / ACCEPT.
Proyecto 1: convertir el AST válido a PigLatin y guardar el archivo .pig.
Proyecto 2: abrir una carpeta completa, editar archivos de los tres lenguajes y guardar cambios.
Proyecto 2: consultar diagnósticos con archivo, línea, columna y mensaje; generar cuartetas, memoria/C3D, C y RISC-V.
Proyecto 2: exportar código C o programa.s y ejecutar el RISC-V generado mediante RARS.

Requisitos y ejecución

Componente	Requisito / utilidad
Sistema operativo	**Windows, Linux o macOS con entorno gráfico.**
Java	**JDK 21 para compilar y ejecutar la aplicación.**
Maven	**Compilación y ejecución del proyecto mediante pom.xml.**
ANTLR4	**Dependencia integrada al proyecto; el proceso Maven genera los analizadores.**
IDE	**IntelliJ IDEA u otro entorno compatible con Maven (opcional para el usuario que recibe el código fuente).**
RARS	**Simulador para ejecutar los archivos ensambladores .s exportados.**
Graphviz	**Herramienta asociada a las gráficas del módulo Codex Latinus del Proyecto 1.**

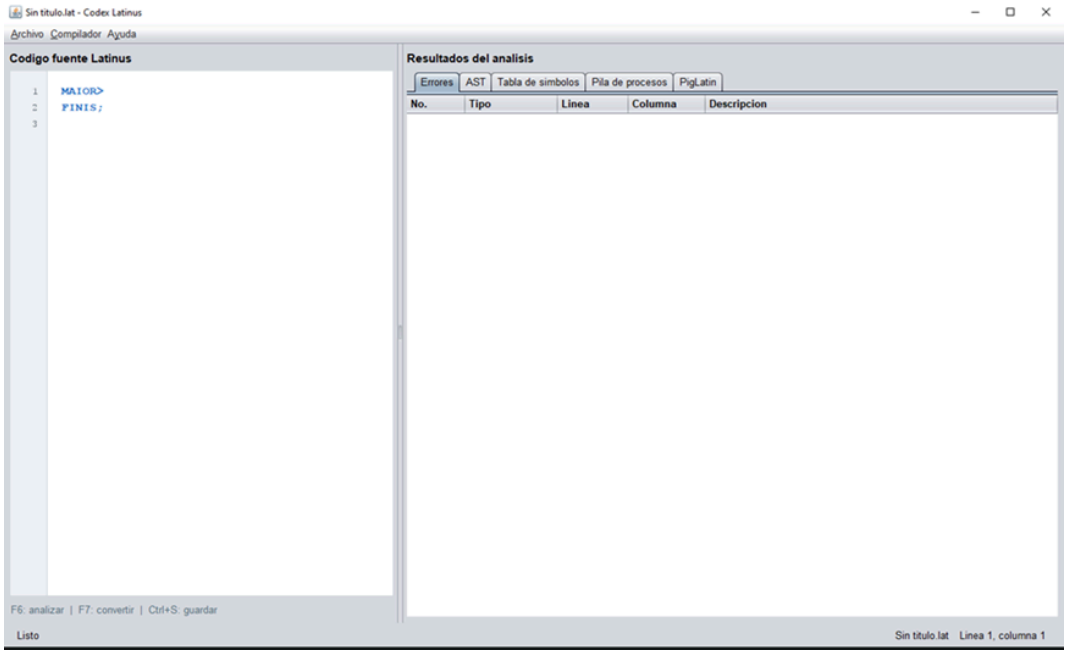
Iniciar la aplicación del Proyecto 2

1. **Ubicarse en la carpeta contacto-extraterrestre, donde se encuentra pom.xml.**
2. **Abrir esa carpeta con IntelliJ IDEA o una terminal que tenga Java 21 y Maven disponibles.**
3. **Ejecutar mvn clean test para verificar la instalación (opcional al abrir la aplicación).**
4. **Ejecutar mvn exec:java para iniciar la interfaz gráfica.**
5. **Desde Archivo > Abrir proyecto..., seleccionar una carpeta que contenga los archivos .pig, .z y/o .y.**

mvn clean test
mvn exec:java

Ventana principal

En Codex Latinus la ventana estaba dividida entre el editor Latinus, un panel de resultados y la barra de estado. Al iniciar, el ejemplo mínimo se representaba con MAIOR> y FINIS; y todavía no aparecían reportes.

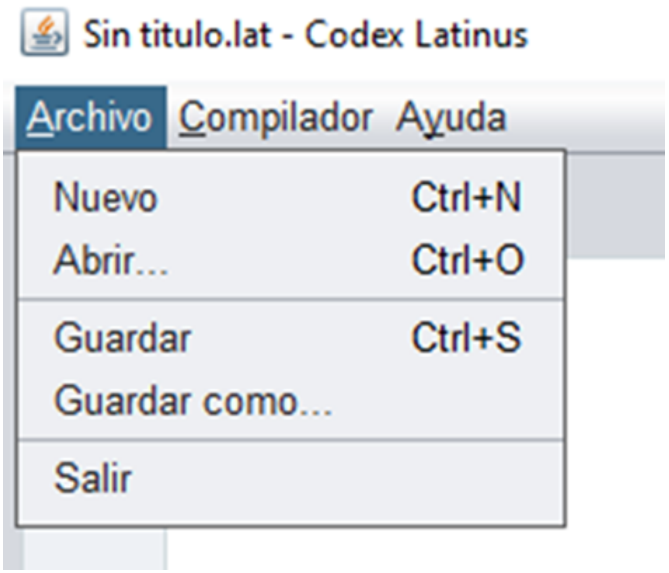


Fotografía 1. Ventana principal de Codex Latinus (Proyecto 1).

Área original	Función
Editor	**Escribir código Latinus con numeración de líneas y coloreado.**
Resultados	**Consultar Errores, AST, Tabla de símbolos, Pila de procesos y PigLatin.**
Barra de estado	**Mostrar el estado del archivo y la ubicación de edición.**
Menú principal	**Acciones Archivo, Compilador y Ayuda.**

Menú Archivo

El menú Archivo del Proyecto 1 permitía crear un archivo .lat, abrirlo, guardar los cambios, guardarlo con otro nombre o salir. Los atajos visibles eran Ctrl+N, Ctrl+O y Ctrl+S.



Fotografía 2. Menú Archivo de Codex Latinus (Proyecto 1).

Opción del Proyecto 1	Uso original
Nuevo	Crear un documento con plantilla MAIOR> / FINIS; y advertencia por cambios sin guardar.
Abrir...	Abrir un archivo .lat existente.
Guardar	Guardar el archivo activo; solicitar nombre si no tenía ruta.
Guardar como...	Guardar una copia con ubicación o nombre distinto.
Salir	Cerrar el programa respetando los cambios pendientes.

Menú Archivo en el Proyecto 2

La interfaz vigente abre proyectos por carpeta. Ofrece Abrir proyecto (Ctrl+O), Guardar archivo (Ctrl+S), Guardar todos (Ctrl+Mayús+S), Descargar archivo seleccionado, Descargar carpeta seleccionada y Salir. El usuario selecciona los archivos desde el árbol del lado izquierdo.

Menú Compilador / Compilación

En el Proyecto 1, el menú Compilador ejecutaba Analizar (F6), Convertir a PigLatin (F7) y Descargar archivo .pig, habilitando la conversión solo después de un análisis válido.



Fotografía 3. Menú Compilador de Codex Latinus (Proyecto 1).

Opción original	Función
Analizar (F6)	Reconocer y validar código Latinus.
Convertir a PigLatin (F7)	Recorrer el AST y generar la traducción.
Descargar archivo .pig	Guardar el programa traducido en UTF-8.

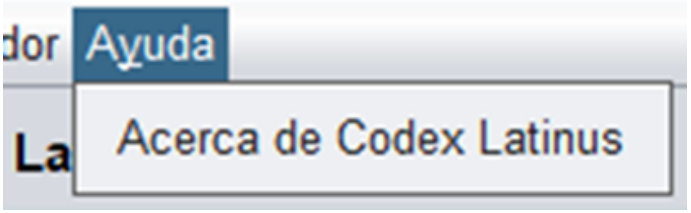
Menú Compilación en el Proyecto 2

En la aplicación actual se utiliza el menú Compilación. Sus opciones son Guardar código C..., Guardar código RISC-V... y Analizar proyecto (F6). La opción de exportación permite guardar el contenido generado, una vez que el análisis termina sin errores y la salida está disponible.

Menú Ayuda y panel de resultados

Menú Ayuda del Proyecto 1

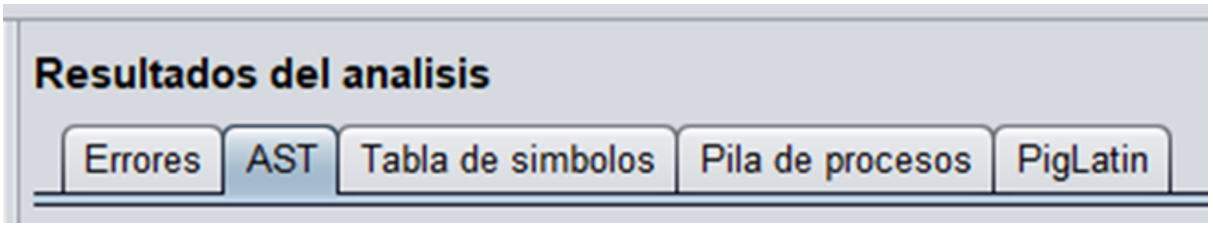
En Codex Latinus, Acerca de Codex Latinus mostraba el nombre del compilador y las tecnologías Java 21, ANTLR4 y Swing. Esta opción aparece en la captura original y no se documenta como menú disponible en la interfaz actual.



Fotografía 4. Menú Ayuda de Codex Latinus (Proyecto 1).

Panel de resultados del Proyecto 1

Tras analizar, el usuario podía seleccionar una de las cinco pestañas: Errores, AST, Tabla de símbolos, Pila de procesos y PigLatin. Cada vista presentaba información independiente.



Fotografía 5. Pestañas del panel de resultados original.

Panel de resultados del Proyecto 2

El Proyecto 2 presenta Diagnósticos, Cuartetas, Memoria / C3D, Código C3D, Código C y Código RISC-V. Los nombres y la función de estas pestañas se explican en las secciones 15 a 18.

Proceso de análisis paso a paso

Procedimiento conservado del Proyecto 1

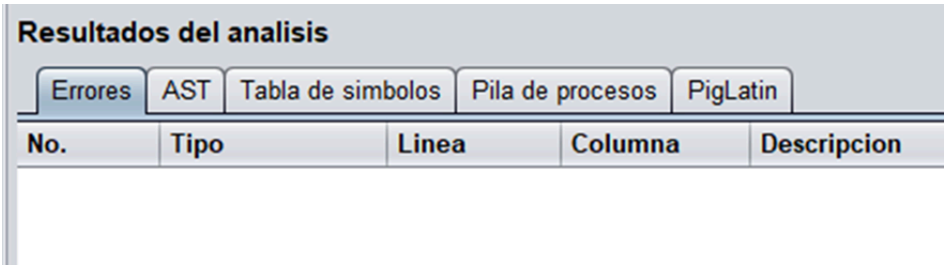
6. Crear o abrir un archivo .lat desde Archivo.
7. Escribir el programa con MAIOR> y FINIS; y guardar con Ctrl+S.
8. Presionar F6 o elegir Compilador > Analizar.
9. Revisar la pestaña Errores; si estaba vacía, consultar AST, tabla de símbolos y pila de procesos.
10. Presionar F7 o Convertir a PigLatin y guardar el archivo .pig cuando se requiriera.

Procedimiento correcto para el Proyecto 2

11. Abrir con Archivo > Abrir proyecto... la carpeta raíz del programa.
12. Seleccionar en el árbol uno de los archivos .pig, .z o .y para revisarlo o editarlo.
13. Guardar el archivo activo con Ctrl+S o utilizar Guardar todos.
14. Presionar F6 o Compilación > Analizar proyecto.
15. Verificar Diagnósticos (0); si hay errores, corregir archivo, línea y columna antes de generar código.
16. Inspeccionar las cuartetas, la representación de memoria y la salida C o RISC-V.
17. Guardar el ensamblador mediante Compilación > Guardar código RISC-V... y ejecutarlo en RARS.

Pestaña Errores / Diagnósticos

En el ejemplo mínimo del Proyecto 1, la tabla Errores no contenía registros. Cuando se detectaba un error, se reportaban el tipo y su localización; el usuario debía corregirlo y volver a analizar.



Fotografía 6. Tabla de Errores vacía en el ejemplo mínimo de Codex Latinus.

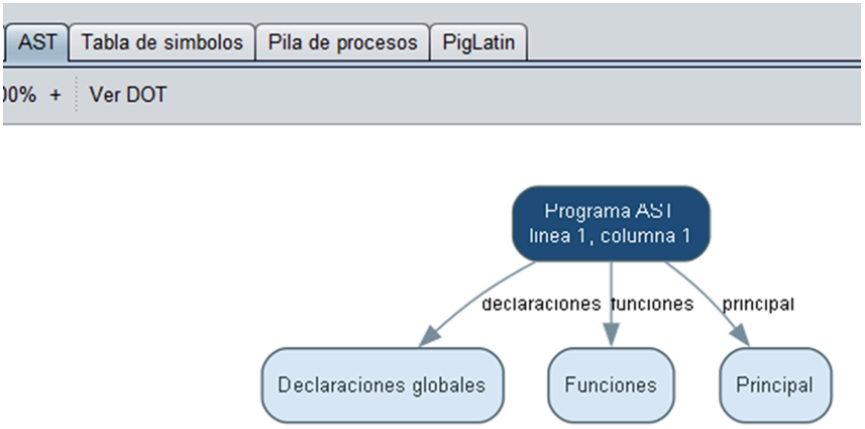
En el Proyecto 2, el equivalente es Diagnósticos. Además de la severidad y tipo, la tabla indica el archivo, línea, columna y mensaje. Un error semántico puede impedir que se produzcan cuartetas o código final.



Captura actual A. Ejemplo de diagnósticos semánticos en Zetariano antes de corregir el banco de pruebas.

Pestaña AST (módulo Codex Latinus)

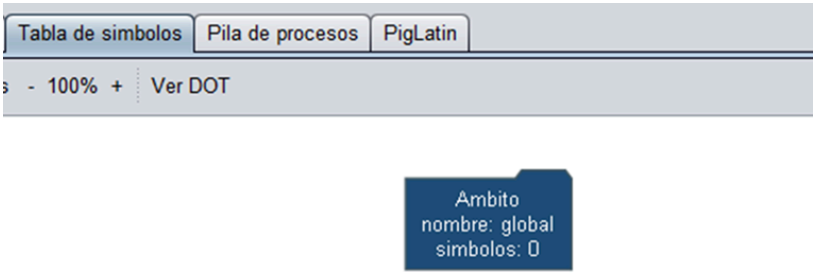
Después de un análisis válido en el Proyecto 1, la pestaña AST mostraba la raíz Programa AST con declaraciones globales, funciones y sección principal. Disponía de controles de zoom y de la opción Ver DOT.



Fotografía 7. AST del programa mínimo en Codex Latinus.

Pestaña Tabla de símbolos (módulo Codex Latinus)

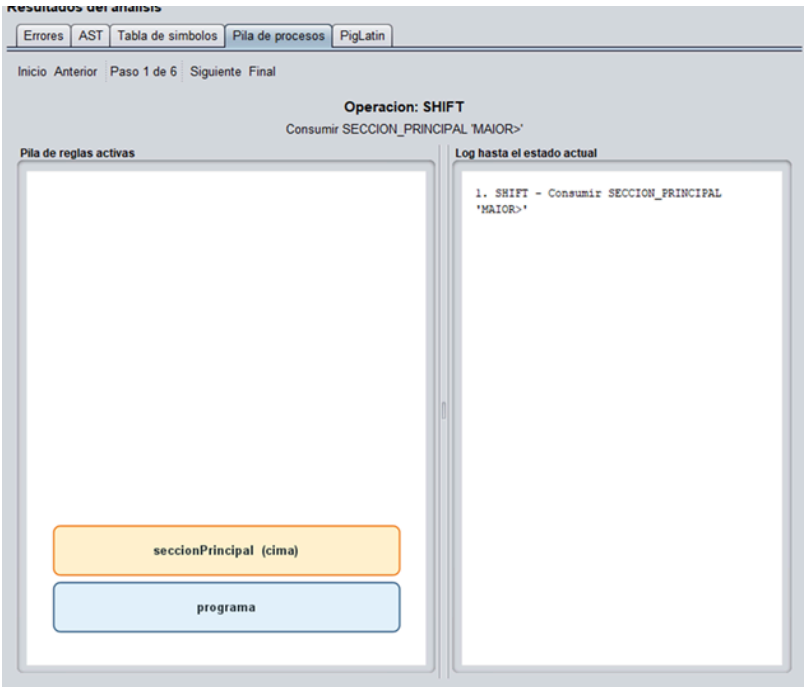
La gráfica de tabla de símbolos permitía consultar los identificadores y sus ámbitos. En el programa mínimo, el ámbito global contenía cero símbolos; al trabajar con programas mayores aparecían variables, arreglos, estructuras, funciones y ámbitos hijos.



Fotografía 8. Ámbito global sin símbolos en Codex Latinus.

Pestaña Pila de procesos (módulo Codex Latinus)

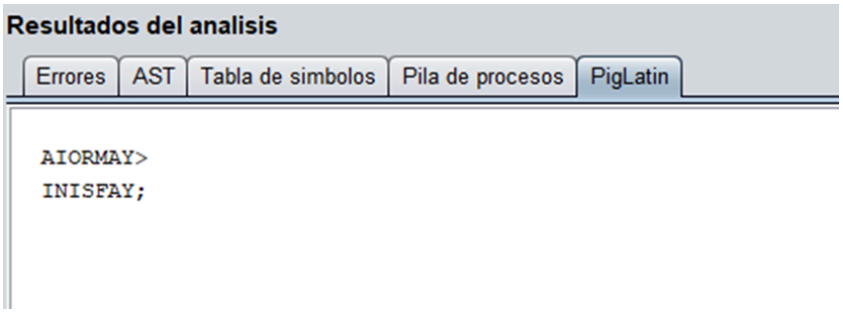
La vista de Pila de procesos permitía recorrer los estados del reconocimiento sintáctico, avanzar o retroceder y revisar el registro de operaciones. Las acciones SHIFT, REPLACE y ACCEPT documentaban el avance del analizador.



Fotografía 9. Pila de procesos y registro de operaciones del Proyecto 1.

Conversión a PigLatin

En el Proyecto 1, la conversión se realizaba después de construir y validar el AST. El resultado se presentaba en la pestaña PigLatin y podía guardarse con extensión .pig. Los identificadores y palabras reservadas se transformaban, mientras los símbolos conservaban su estructura.



Fotografía 10. Resultado de conversión del programa mínimo a PigLatin.

Reglas visibles de traducción

Elemento	Ejemplo
Consonantes iniciales	**fuerza → uerzafay**
Vocales iniciales	**inicio → inicioway**
Lectura	**<< → %OINK_OINK**
Impresión	**>> → %OINK**

Ejemplo mínimo completo

Código Latinus (Proyecto 1)

```
MAIOR>
FINIS;
```

Resultado PigLatin documentado

```
AIORMAY>
INISFAY;
```

Este ejemplo se conserva para mostrar la traducción del Proyecto 1. En el Proyecto 2 el usuario normalmente abre una carpeta con un archivo principal .pig y, cuando corresponda, clases .z y bibliotecas .y.

Atajos de teclado

Atajos del manual original

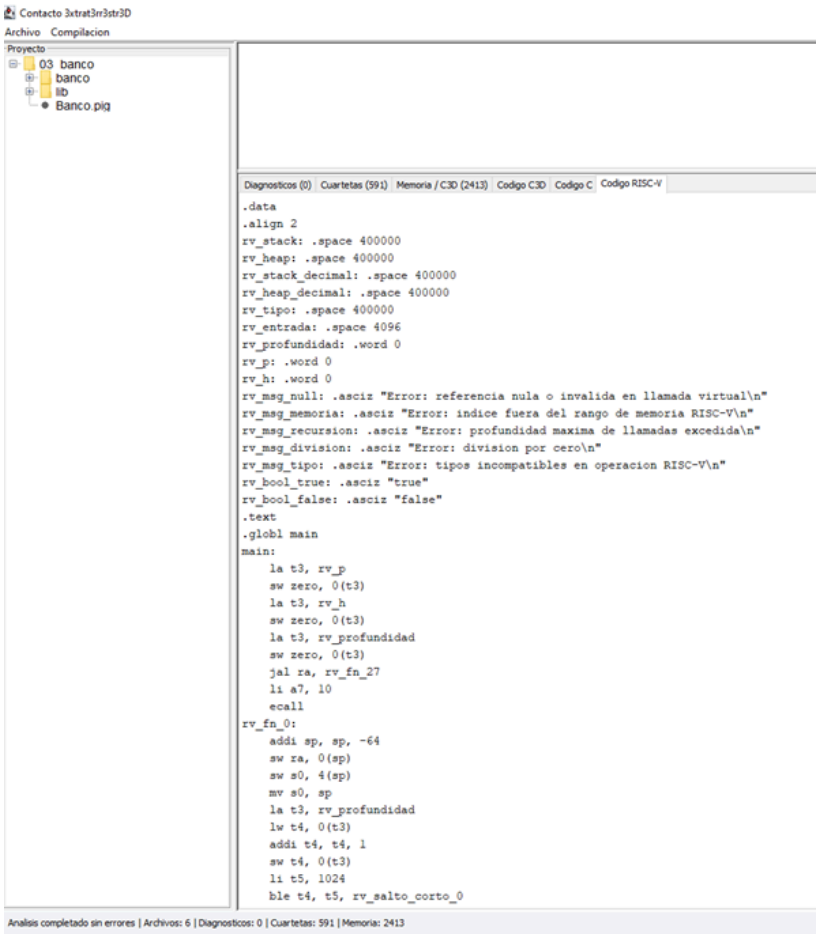
Atajo	Acción en Codex Latinus
Ctrl+N	Crear un archivo Latinus.
Ctrl+O	Abrir un archivo .lat.
Ctrl+S	Guardar el archivo actual.
F6	Analizar Latinus.
F7	Convertir a PigLatin.

Atajos actuales verificados en el Proyecto 2

Atajo	Acción actual
Ctrl+O	Abrir una carpeta de proyecto.
Ctrl+S	Guardar el archivo abierto.
Ctrl+Mayús+S	Guardar todos los archivos abiertos.
F6	Analizar el proyecto completo.

Interfaz actual: árbol del proyecto y editor

Al abrir una carpeta, la aplicación muestra a la izquierda el árbol del proyecto con subcarpetas y archivos reconocidos. Para editar, se selecciona un archivo .pig, .y o .z; al analizar, el resultado se publica en las pestañas inferiores.



Captura actual B. Banco de la auxiliar analizado: árbol de archivos, cero diagnósticos y ensamblador disponible.

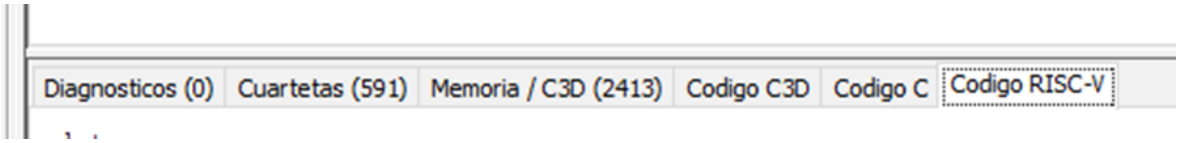
Gestión de carpetas y archivos

Abrir proyecto...: carga una carpeta y su árbol.
Guardar archivo: conserva los cambios del editor activo.
Guardar todos: aplica los cambios pendientes a los archivos abiertos.
Descargar archivo seleccionado o carpeta seleccionada: guarda una copia en la ubicación elegida.

Interpretar el análisis del Proyecto 2

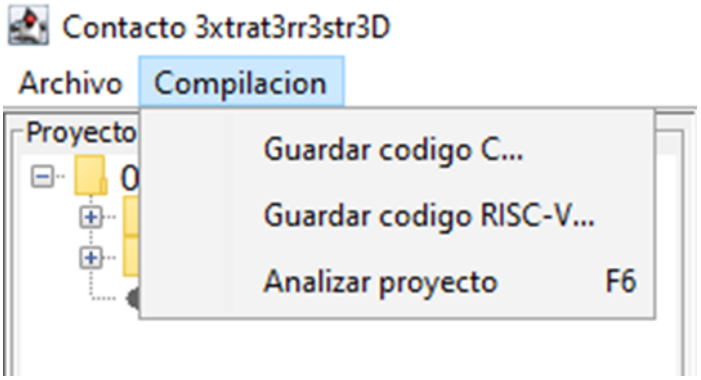
Una ejecución correcta puede mostrar valores como Diagnósticos (0), Cuartetas (591) y Memoria / C3D (2413) al analizar el banco de la auxiliar. Los conteos dependen del programa; no existe un número fijo de cuartetas para cualquier entrada.

Pestaña	¿Qué presenta?
Diagnósticos	Errores y advertencias con archivo, línea, columna y mensaje.
Cuartetas	Operaciones intermedias en una representación de cuatro campos.
Memoria / C3D	Instrucciones para variables, Stack, Heap y llamadas.
Código C3D	Traducción del código intermedio a instrucciones legibles.
Código C	Programa C generado para compilación y comprobaciones adicionales.
Código RISC-V	Ensamblador listo para exportar cuando la generación es satisfactoria.



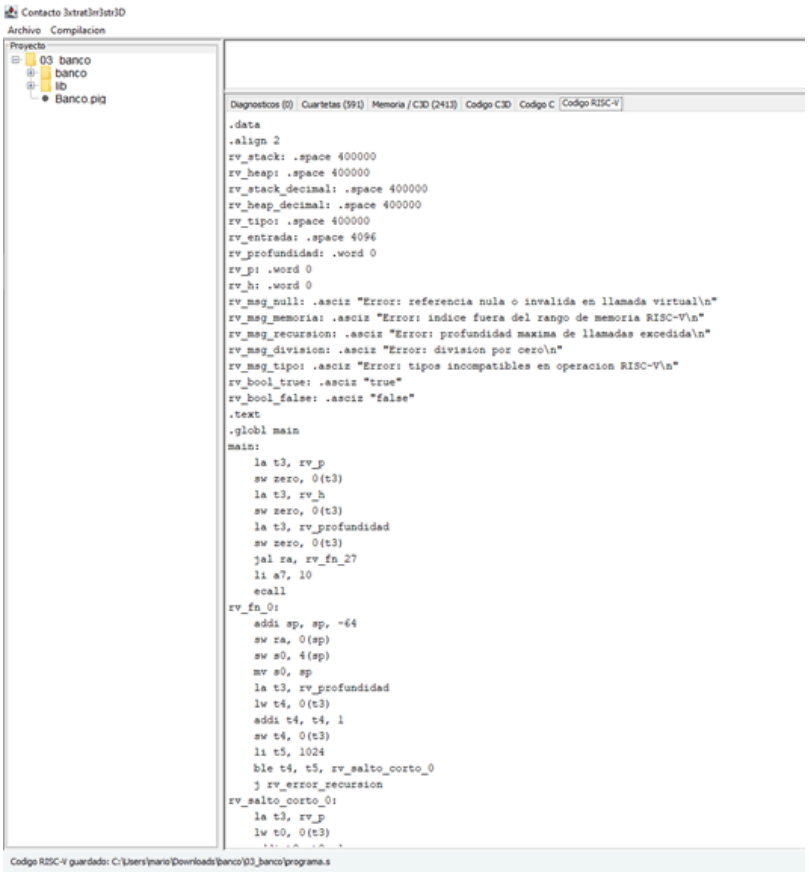
Fotografía adicional 12. Pestañas actuales, incluyendo Código RISC-V.

Consultar el código RISC-V



Fotografía adicional 11. Menú Compilación del Proyecto 2 con exportación de C y RISC-V.

La pestaña Código RISC-V muestra las secciones de datos y texto del ensamblador. En los ejemplos validados aparecen etiquetas de memoria, instrucciones de control, operaciones sobre Stack y Heap y servicios de impresión o salida.

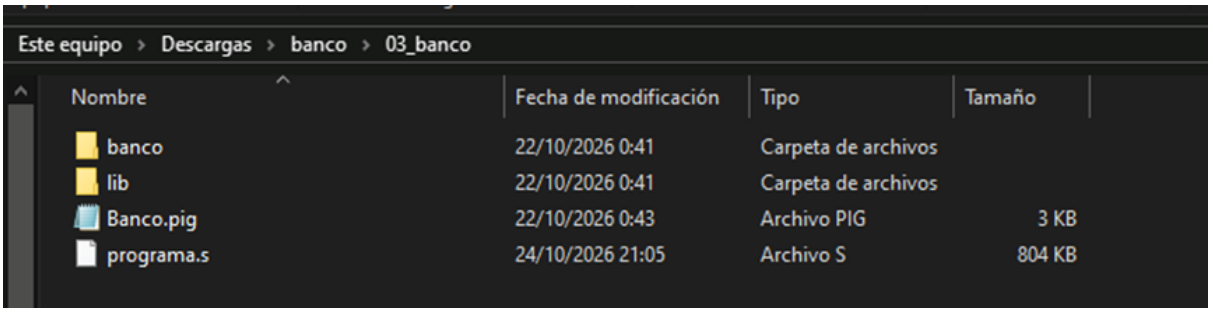


Fotografía adicional 13. Generación RISC-V del banco de pruebas, con secciones .data y .text.

Nota. La presencia de texto ensamblador no basta para asegurar que un programa es correcto: es necesario guardarlo y ejecutarlo en RARS para verificar los resultados.

Guardar el ensamblador

18. Analizar la carpeta y confirmar que Diagnósticos indique cero errores.
19. Seleccionar la pestaña Código RISC-V.
20. Abrir Compilación > Guardar código RISC-V...
21. Elegir la carpeta deseada y guardar el archivo con nombre programa.s.
22. Comprobar en el explorador que el archivo .s esté junto al proyecto que lo generó.



Fotografía adicional 14. Carpeta del banco que contiene programa.s.

Ejecutar programa.s en RARS

Con Java instalado y el simulador rars1_6.jar disponible en Descargas, abre PowerShell en la carpeta donde guardaste programa.s. Ejecuta el siguiente comando:

java -jar "$env:USERPROFILE\Downloads\rars1_6.jar" nc .\programa.s



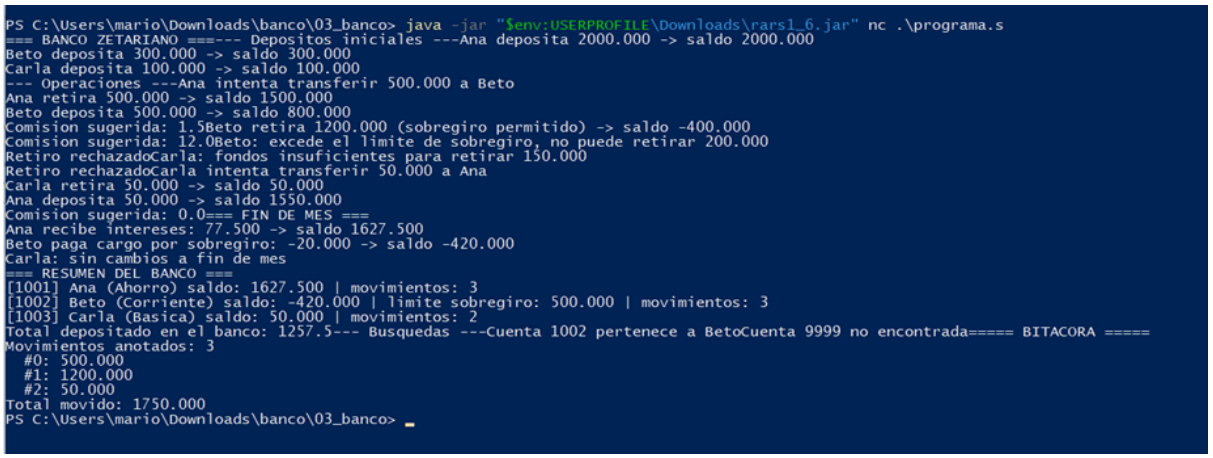
Fotografía adicional 15. Comando de PowerShell para ejecutar programa.s con RARS.

Si el ensamblador se ejecuta correctamente, la consola muestra la salida del programa. Si RARS reporta un error con número de línea, revisa el archivo exportado o vuelve a generarlo desde la última versión del proyecto.

Prueba integral: banco de la auxiliar

El banco íntegra código PigLatin, clases Zetariano y una biblioteca Y?. Después de corregir el atributo heredado movimientos como protected en el archivo proporcionado, se analizó sin diagnósticos y se generó el ensamblador para RARS.

La salida comprueba depósitos, retiros, transferencias, sobregiro, intereses, búsqueda de cuentas y una bitácora de movimientos. Estos resultados son una evidencia funcional de ejecución; no reemplazan las pruebas de otros programas.



Fotografía adicional 16. Salida del banco de pruebas en PowerShell usando RARS.

Saldos comprobados al finalizar

Cuenta	Saldo final
Ana (Ahorro)	**1627.500**
Beto (Corriente)	**-420.000**
Carla (Básica)	**50.000**
Suma de saldos	**1257.500**

Recomendaciones de uso

Guardar los archivos antes de ejecutar un análisis importante.
Corregir los errores reportados desde las líneas más tempranas para evitar diagnósticos encadenados.
Volver a analizar después de modificar cualquier archivo del proyecto.
No confundir la generación de código con su ejecución: validar programa.s en RARS.
Conservar una copia del código fuente y del ensamblador empleado en las evidencias.
Si se usa un ejemplo del Proyecto 1, recordar que sus menús y teclas de acceso no son idénticos a los del Proyecto 2.

Problemas frecuentes y solución

Situación	Acción recomendada
La ventana muestra “Sin proyecto”	**Abrir una carpeta desde Archivo > Abrir proyecto...**
No aparece código RISC-V	**Revisar Diagnósticos, corregir errores y volver a analizar.**
Se detecta un atributo heredado no accesible	**Comprobar public, private, protected y la regla default indicada para el proyecto.**
El archivo .s no se encuentra	**Guardar el ensamblador y abrir PowerShell en la carpeta correcta.**
RARS no abre	**Revisar instalación de Java y ruta de rars1_6.jar.**
RARS muestra error de ensamblado	**Reexportar desde el generador corregido y revisar el mensaje y número de línea.**
Los números salen unidos en consola	**Revisar si las instrucciones de salida incluyen saltos de línea; no indica por sí solo error aritmético.**